

Investigating the relationship between Netflix catalog and critical ratings

Tuan Anh Mirko Nguyen Hoang

December 30, 2025

Abstract

This study presents the findings of a statistical analysis conducted within the RStudio environment, designed to investigate potential correlations between the Netflix production catalog and aggregated reviews from both Rotten Tomatoes and IMDb.

Chapter 1 introduces the selected datasets and outlines the rationale behind their selection.

Chapter 2 details the data cleaning methodologies and the processing techniques employed to merge the different sources into a unified dataframe for subsequent analysis.

Chapter 3 examines the Exploratory Data Analysis (EDA) applied to identify initial correlations and distinct characteristics within the individual datasets.

Chapter 4 presents the same operations of cleaning/merging and EDA applied on the new IMDB dataset.

Chapter 5 shows the operations we used to find the key correlations between the 3 datasets.

Chapter 6 concludes with a summary of the analysis and discusses the limitations encountered.

Contents

1	Description of the Datasets	3
1.1	Rationale for Selection	3
1.2	Data Sources	3
1.3	Preliminary analysis of the Netflix dataset	4
1.4	Preliminary analysis for the Rotten Tomatoes dataset	4
1.5	Preliminary analysis for the IMDB dataset	5
2	Data cleaning and preprocessing	6
2.1	Cleaning and trasformation of the netflix dataset	6
2.1.1	Counting of NA	6
2.1.2	Replacement of NA values	7
2.1.3	Expansion of multi-valued records	7
2.1.4	Normalization of critic's score	7
2.1.5	Filtering of TV shows	8
2.1.6	Standardization of the length	8
2.1.7	Aggregation of category	9
2.2	Preparation for the join	9
2.2.1	Standardization of the movie title	9
2.2.2	Aggregation of reviews	10
2.3	Join between Netflix movies and Rotten Tomatoes datasets	10
3	Exploratory Data Analysis (EDA)	12
3.1	EDA for the Rotten Tomatoes dataset	12
3.1.1	Sentiment count	12
3.1.2	Sentiment mean	13
3.1.3	Count of freshness	14
3.2	EDA for the joined dataset (Netflix movies + Rotten Tomatoes) . .	15
3.2.1	Distribution of mean scores	15
3.2.2	Mean scores for genre	16
3.2.3	Mean scores for release year	17
3.2.4	Top directors analysis	18
3.2.5	Scores and length analysis	19
3.2.6	Country and scores analysis	20
3.2.7	Rating and scores correlation	23
3.2.8	Actors and scores analysis	23

4	IMDB dataset	25
4.1	Motivations	25
4.2	Data cleaning	25
4.2.1	Na count and replacement	25
4.2.2	Normalization of the scores	26
4.2.3	Normalization of the date	26
4.3	Preparation for the join operation	26
4.4	EDA for the IMDB dataset	27
4.4.1	frequency of the average scores	27
4.4.2	Temporal distribution of the scores	28
5	Final dataset analysis	30
5.1	Na control	30
5.2	EDA for the final dataset	31
5.2.1	Top films with different scores	31
5.2.2	Top genres with different scores	31
5.2.3	Genres and origin country correlation	32
5.2.4	Length sweet spot difference	33
6	Conclusion	35
6.1	Summary of findings	35
6.2	Limitations of the study	35

Chapter 1

Description of the Datasets

1.1 Rationale for Selection

The primary objective of this study is to identify and examine significant correlations between specific cinematographic attributes—such as duration, genre, country of origin, cast, and direction—and the resulting reception across different platforms. By distinguishing between critical acclaim and user satisfaction, this analysis seeks to highlight potential divergences or patterns in how various types of content are perceived.

1.2 Data Sources

The present analysis utilizes three distinct datasets sourced from the Kaggle repository. These datasets provide the foundational information required to cross-reference streaming content with audience and critical reception:

- **Netflix Movies and TV Shows:** This dataset comprises comprehensive metadata regarding the titles available on the platform.
 - **Number of records:** 8807.
 - **Columns:** 12.
- **Rotten Tomatoes Reviews:** This source provides scores and qualitative reviews from both professional critics and the general public.
 - **Number of records:** 1,444,963.
 - **Columns:** 11.
- **IMDb Reviews:** This dataset focuses on user-generated ratings and reviews, offering a broader perspective on global audience sentiment.
 - **Number of records:** 20311.
 - **Columns:** 8.

1.3 Preliminary analysis of the Netflix dataset

- **title:** The title of the content as it appeared at the time of publication.
- **type:** Categorization of the entry as either a *Movie* or a *TV Show*.
- **director / cast / listed_in:** Multi-valued fields that may contain multiple entries (e.g., several actors or genres) separated by commas.
- **date_added:** The date the title was added to the platform, recorded in the "Month Day, Year" format.
- **release_year:** The original year of publication or broadcast.
- **rating:** A categorical variable representing various content maturity ratings.
- **duration:** Quantitative measurement expressed in minutes for movies and in number of seasons for TV shows.
- **other columns:** Variables that were neither utilized nor modified during the data processing phase.

1.4 Preliminary analysis for the Rotten Tomatoes dataset

- **id:** A unique identifier based on the film title. In its raw form, it may include an 8-digit numerical string and uses underscores (_) instead of standard spacing.
- **creationDate:** The date the review was published, recorded in the "YYYY-MM-DD" format.
- **isTopCritic:** A boolean variable used to distinguish reviews written by high-tier or "Top" critics from general reviewers.
- **originalScore:** The raw rating assigned by the critic. This field exhibits high variability in format, including fractional scales (e.g., x/5, x/10, x/4) and alphabetical grades (e.g., "A").
- **reviewState:** A categorical attribute indicating the current status of the review (e.g., identifying recent entries).
- **scoreSentiment:** A categorical variable representing the overall tone or sentiment of the review (e.g., positive or negative).
- **other columns:** Additional metadata and variables that were neither utilized nor modified during the data processing phase.

1.5 Preliminary analysis for the IMDB dataset

- **movie:** The title of the content as it appeared at the time of publication.
- **rating:** The value of the rating of the review, in the x/10 format.
- **date:** The date on which the film was reviewed, recorded in the "Month Day, Year" format.
- **other columns:** Additional metadata and variables that were neither utilized nor modified during the data processing phase.

Chapter 2

Data cleaning and preprocessing

2.1 Cleaning and transformation of the netflix dataset

2.1.1 Counting of NA

The analysis focuses on the columns that we have highlighted previously, transforming them in a suitable format. Before doing that we count how many NA values the dataset has, to do that we used this code:

```
1 na_alternativo_netflix <- netflix %>%  
2   summarise(across(everything(), ~ sum(is.na(.)) / n() * 100)) %>%  
3   t() %>%  
4   as.data.frame() %>%  
5   rownames_to_column(var = "Colonna") %>%  
6   rename(NA_Percentuale = V1) %>%  
7   mutate(NA_Percentuale = round(NA_Percentuale, 2)) %>%  
8   arrange(desc(NA_Percentuale))
```

From here, we can see the percentage of NA values for each column.

	Colonna	NA_Percentuale
1	director	29.91
2	country	9.44
3	cast	9.37
4	date_added	0.11
5	rating	0.05
6	duration	0.03
7	show_id	0.00
8	type	0.00
9	title	0.00
10	release_year	0.00
11	listed_in	0.00
12	description	0.00

Figure 2.1: Distribution of NA values

2.1.2 Replacement of NA values

After counting how many NA values we had, we proceeded to remove them with the "unknown" string:

```
1 # Gestione dei valori NA nella colonna 'director'
2 netflix_pulito <- netflix %>%
3   mutate(director = replace_na(director, "Unknown"))
```

For the **rotten_reviews["originalScore"]**, **netflix["country"]**, **netflix["cast"]** a similar procedure was applied, but instead of replacing we directly eliminated the records containing NA values.

2.1.3 Expansion of multi-valued records

for the **netflix["listed_in"]** we opted to divide the individual records with multiple values in multiple records each with a single value, using this code:

```
1 netflix_pulito_espanso <- netflix_pulito %>%
2   separate_rows(listed_in, sep = ", ") %>%
3   mutate(listed_in = trimws(listed_in))
```

We noticed that the number of records almost tripled (from 8807 to 19323)

2.1.4 Normalization of critic's score

As we talked about before, the column **rotten_reviews["originalScore"]** is recorded in various formats, so we decided to transform them in a "(1-10)" format, to ensure that scaling issues are avoided later on, to do that we first created a mapping table, where we assigned a value to all possible letters used to score:

```
1 letter_to_score <- c(
2   "A+" = 10, "A" = 9.5, "A-" = 9.0,
3   "B+" = 8.5, "B" = 8.0, "B-" = 7.5,
4   "C+" = 7.0, "C" = 6.5, "C-" = 6.0,
5   "D+" = 5.5, "D" = 5.0, "D-" = 4.5,
6   "F" = 3.0, "F-" = 2.0, "F+" = 4.0
7 )
```

Then we treated the transformation of every single case:

```
1 rotten_pulito_normalizzato <- rotten_pulito %>%
2   mutate(
3     normalizedScore = NA_real_,
4     score_text = as.character(originalScore)
5   ) %>%
6
7   mutate(
8     normalizedScore = case_when(
9       #Controlla se il valore corrisponde a un voto nella tabella (A, B+, C-,
      ecc.)
10     score_text %in% names(letter_to_score) ~ letter_to_score[score_text],
```

```

11
12 #Voto frazionario
13 str_detect(score_text, "/") ~ {
14   parts <- str_split_fixed(score_text, "/", n = 2)
15   numerator <- as.numeric(parts[, 1])
16   denominator <- as.numeric(parts[, 2])
17
18   (numerator / denominator) * 10
19 },
20
21 #Voto gia su scala 1-10
22 !is.na(as.numeric(score_text)) & as.numeric(score_text) <= 10 ~ as.
numeric(score_text),
23
24 #Valori mancanti
25 TRUE ~ NA_real_
26 )
27 ) %>%
28 select(-score_text)
29
30 rotten_pulito_normalizzato_finale <- rotten_pulito_normalizzato %>%
31 drop_na(normalizedScore)

```

2.1.5 Filtering of TV shows

We used the **netflix["type"]** column to filter out all of the records representing "TV show":

```

1 netflix_film_pulito <- netflix_pulito_espanso %>%
2   filter(type == "Movie") %>%
3   select(
4     titolo_film = title,
5     anno_rilascio = release_year,
6     listed_in,
7     director,
8     cast,
9     country,
10    duration,
11    rating,
12    type
13  )

```

We noticed that roughly 30% of the records were about "TV show" (from 19323 to 13190)

2.1.6 Standardization of the length

For the **netflix["duration"]** we transformed all the records that contained the "min" string:

```

1 netflix_film_pulito_finale <- netflix_film_pulito %>%
2   mutate(duration_min = as.numeric(str_remove(duration, " min"))) %>%

```

```
3 select(-duration)
```

2.1.7 Aggregation of category

To facilitate working with the various pg-ratings we used the **netflix["rating"]** column, and aggregated the various values in three macro categories:

```
1 netflix_film_pulito_finale <- netflix_film_pulito_finale %>%
2   mutate(rating_semplificato = case_when(
3     rating %in% c("G", "TV-Y", "TV-G") ~ "Bambini",
4     rating %in% c("PG", "TV-PG", "PG-13") ~ "Adolescenti/Famiglia",
5     rating %in% c("R", "NC-17", "TV-MA") ~ "Adulti",
6     TRUE ~ "Non Definito"
7   ))
```

2.2 Preparation for the join

2.2.1 Standardization of the movie title

We initially worked with the Netflix and Rotten Tomatoes datasets, and to prepare them for the join operation, we had to modify some columns.

First the movie title column for the Netflix dataset (**netflix["titolo_film"]**):

```
1 library(stringr)
2 netflix_join <- netflix_film_pulito_finale %>%
3   mutate(
4     # 1. Converta il titolo originale in minuscolo
5     titolo_normalizzato = tolower(titolo_film),
6
7     #Sostituisci tutti i caratteri che NON sono lettere minuscole o numeri (a-z, 0-9)
8     titolo_normalizzato = str_replace_all(titolo_normalizzato, "[^a-z0-9]", " "),
9     titolo_normalizzato = str_remove_all(titolo_normalizzato, " "),
10
11     #Rimuovi spazi
12     titolo_normalizzato = str_squish(titolo_normalizzato)
13   )
```

Then the id column for the Rotten Tomatoes dataset:

```
1 rotten_join <- rotten_pulito_normalizzato_finale %>%
2   mutate(
3
4     titolo_normalizzato = tolower(id),
5
6     # Rimuove sequenze di cifre lunghe all'inizio della stringa (es. '987654
7     # titolo')
8     titolo_normalizzato = str_remove(titolo_normalizzato, "^\\d{6,}\\s*"),
9
10    # Rimuove pattern come ' (2001)' o '[2001]' ovunque compaiano
```

```

10 titolo_normalizzato = str_remove_all(titolo_normalizzato, "[\\s\\(\\[\\]\\d
    {4}\\(\\)\\[\\]"]"),
11
12 # Cerca un anno di 4 cifre (19xx o 20xx) alla fine della stringa
13 titolo_normalizzato = str_remove(titolo_normalizzato, "\\d{4}$"),
14
15 #Rimuove la maggior parte della punteggiatura (sostituisce con uno spazio)
16 titolo_normalizzato = str_replace_all(titolo_normalizzato, "[^a-z0-9]", " ")
17
18 #Rimuove spazi
19 titolo_normalizzato = str_squish(titolo_normalizzato),
20 titolo_normalizzato = str_remove_all(titolo_normalizzato, " ")
21 )%>%
22 filter(titolo_normalizzato != "")

```

2.2.2 Aggregation of reviews

Before joining we have to aggregate and calculate the average score for each film in the Rotten Tomatoes dataset:

```

1 rotten_aggregato <- rotten_join %>%
2   group_by(titolo_normalizzato) %>%
3   summarise(
4     media_score_rotten = mean(normalizedScore, na.rm = TRUE),
5     conteggio_recensioni = n()
6   ) %>%
7   ungroup()

```

2.3 Join between Netflix movies and Rotten Tomatoes datasets

After all of these preparations, the final join operation becomes really easy:

```

1 dataset_unit0 <- netflix_join %>%
2   left_join(rotten_aggregato,
3     by = "titolo_normalizzato")

```

After the union of the datasets we can check how many films appear in only one of the datasets:

```

1 dataset_finale <- dataset_unit0 %>%
2   # Rimuove tutte le righe che hanno NA nella colonna della media dello score
   Rotten.
3   drop_na(media_score_rotten)
4
5 film_rimasti <- nrow(dataset_finale)
6 film_rimossi <- nrow(dataset_unit0) - film_rimasti
7
8 cat("--- Verifica Filtro Recensioni ---\n")

```

```
9 cat("Record totali nel dataset unito (Netflix):", nrow(dataset_unitito), "\n")
10 cat("Film rimossi (senza corrispondenza Rotten):", film_rimossi, "\n")
11 cat("Film nel dataset finale per l'analisi:", film_rimasti, "\n")
```

After performing a data merging process and removing non-matching entries, the dataset was refined from an initial 13,190 records to a final sample of 7,350.

Chapter 3

Exploratory Data Analysis (EDA)

3.1 EDA for the Rotten Tomatoes dataset

3.1.1 Sentiment count

The initial phase of the analysis involved quantifying the sentiment across the review dataset. To facilitate this, sentiment categories were mapped to numerical values:

```
1 # Mappatura del Sentiment a un valore numerico
2 rotten_sentiment <- rotten_join %>%
3   mutate(
4     sentiment_score_num = case_when(
5       scoreSentiment == "POSITIVE" ~ 1,
6       scoreSentiment == "NEGATIVE" ~ -1,
7       scoreSentiment == "NEUTRAL" ~ 0,
8       TRUE ~ NA_real_
9     )
10  )
```

Listing 3.1: Numerical mapping of sentiment categories

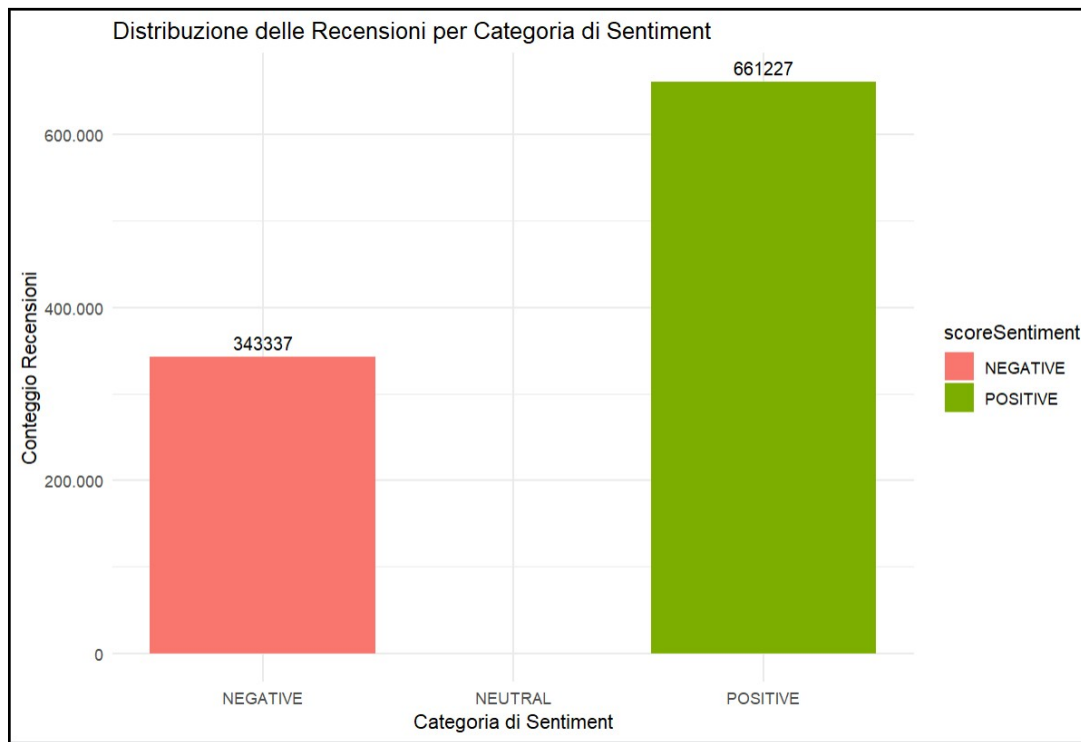


Figure 3.1: Distribution of reviews based on sentiment categories

3.1.2 Sentiment mean

Subsequently, the aggregate mean sentiment was computed to determine the overall emotional tone of the sample:

```
1 # Calcolo del Sentiment Medio
2 media_sentiment_totale <- rotten_sentiment %>%
3   summarise(
4     Sentiment_Medio = mean(sentiment_score_num, na.rm = TRUE)
5   )
```

Listing 3.2: Computation of the aggregate mean sentiment

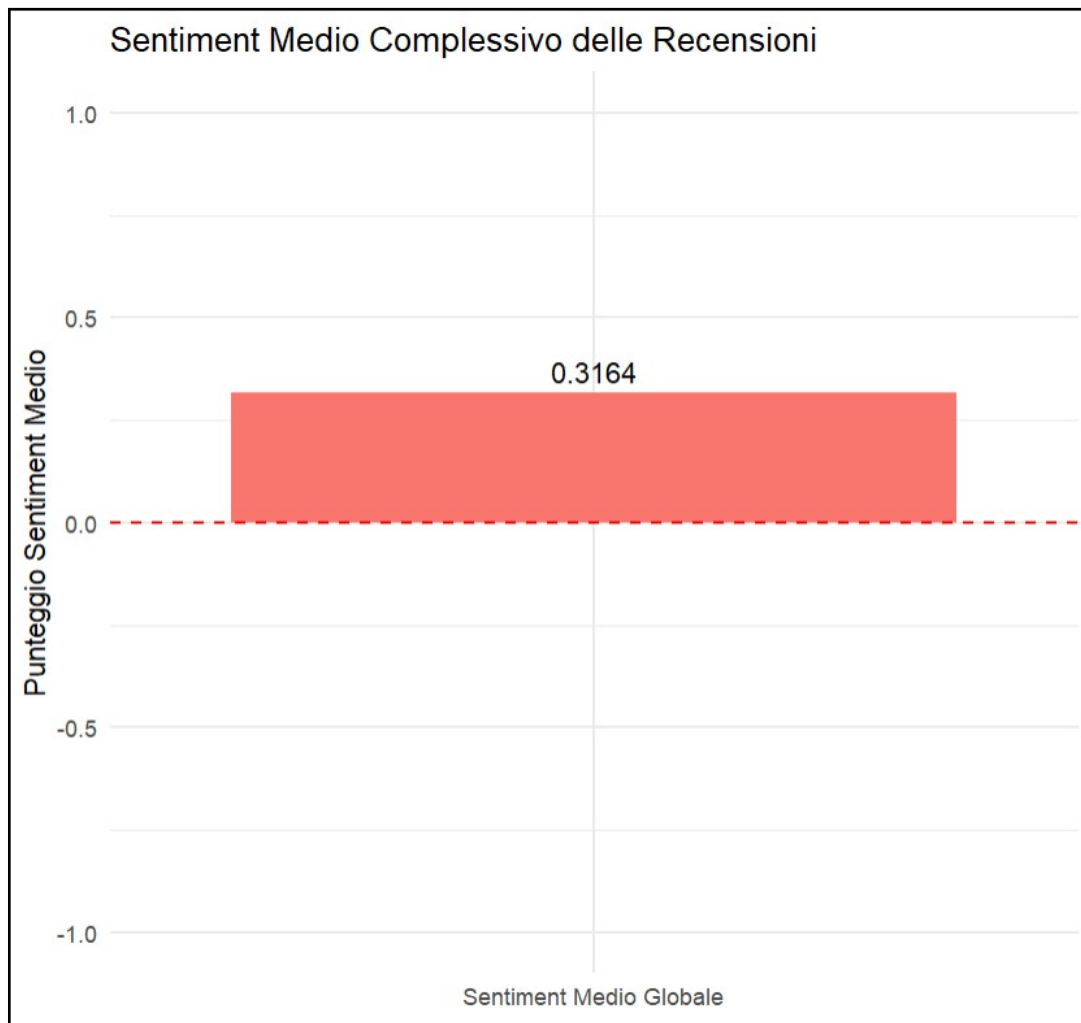


Figure 3.2: Visual representation of the general mean sentiment

3.1.3 Count of freshness

Similarly, the distribution of review "freshness" (review state) was analyzed:

```
1 conteggio_recensioni <- rotten_join %>%
2   group_by(reviewState) %>%
3   summarise(
4     Conteggio = n()
5   ) %>%
6   ungroup()
```

Listing 3.3: Computation of freshness count

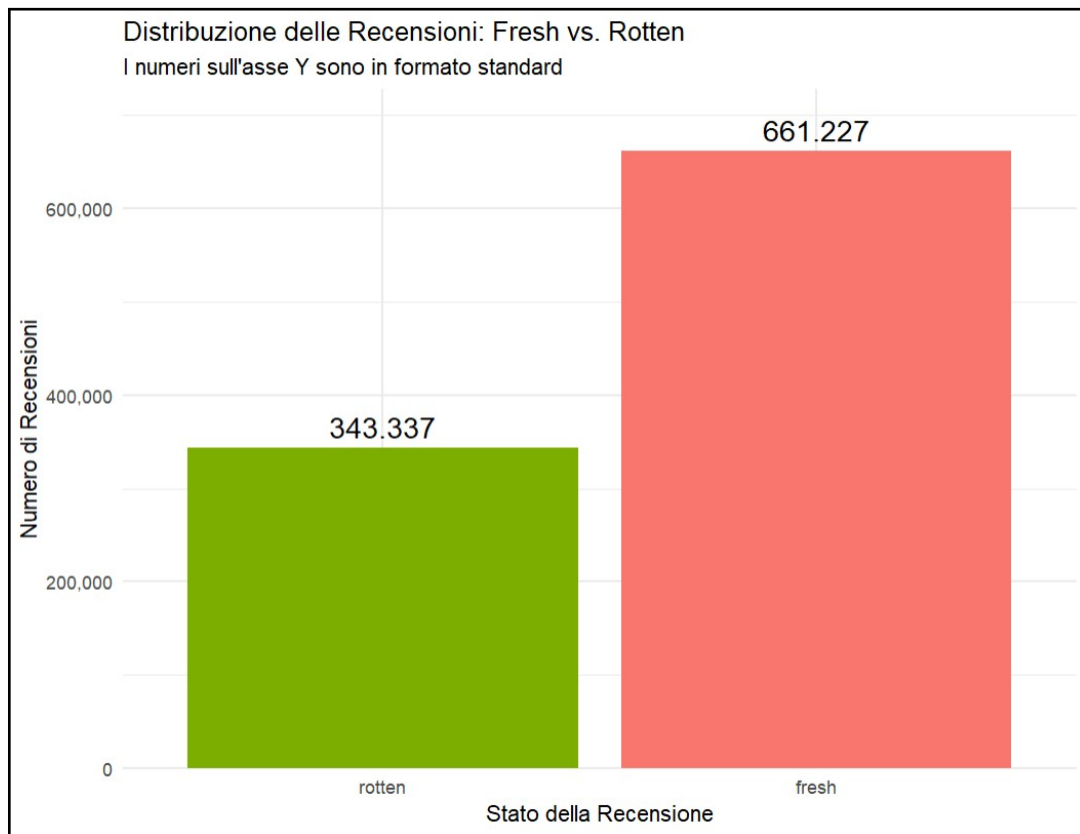


Figure 3.3: Distribution of the freshness of the reviews

3.2 EDA for the joined dataset (Netflix movies + Rotten Tomatoes)

3.2.1 Distribution of mean scores

In this part we grouped up and plotted the distribution of the mean scores for the various films:

```
1 ggplot(dataset_finale, aes(x = media_score_rotten)) +
2   geom_histogram(binwidth = 0.5, fill = "lightblue", color = "black") +
3   labs(
4     title = "Distribuzione dei Punteggi Medi Rotten Tomatoes",
5     x = "Punteggio Medio Normalizzato (0-10)",
6     y = "Frequenza (Numero di Film)"
7   ) +
8   theme_minimal()
```

Listing 3.4: Mean scores plotting

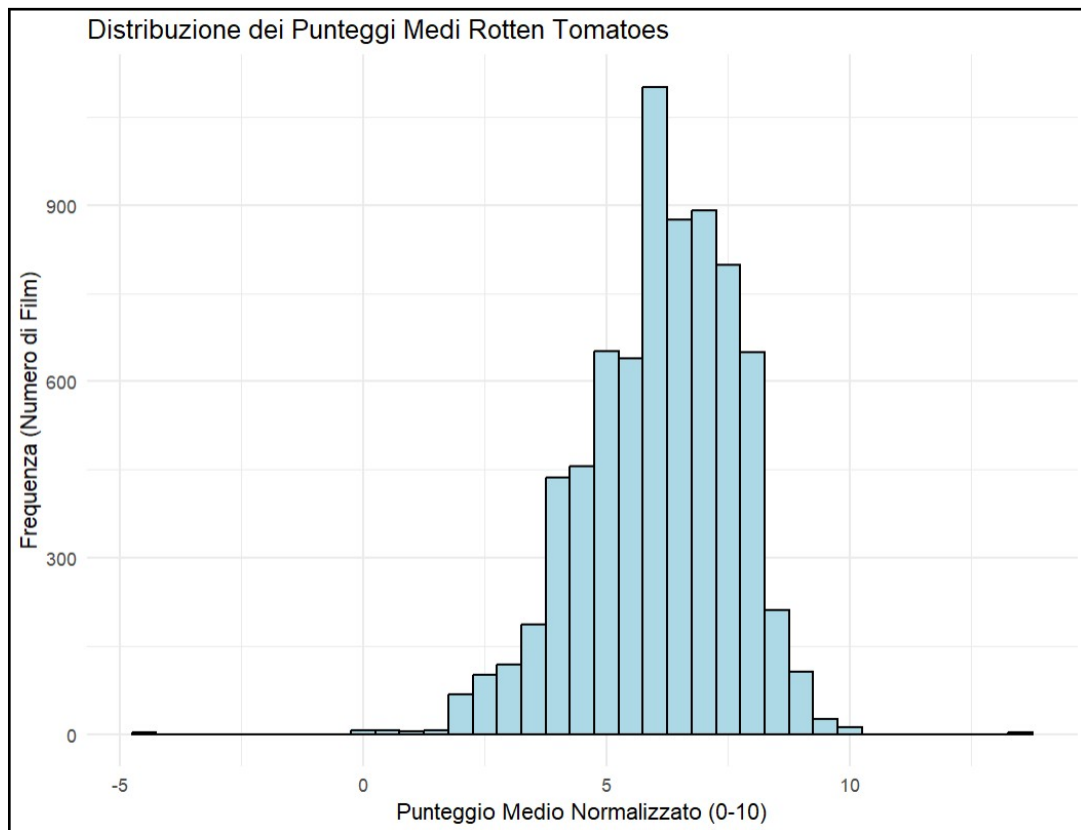


Figure 3.4: Distribution of the mean scores

3.2.2 Mean scores for genre

In order to find the mean scores for each genre we first have to group by **["listed_in"]**:

```
1 punteggi_per_genere <- dataset_finale %>%
2   group_by(listed_in) %>%
3   summarise(
4     punteggio_medio = mean(media_score_rotten, na.rm = TRUE),
5     totale_film = n()
6   ) %>%
7   # Filtra i generi con poche osservazioni
8   filter(totale_film >= 50) %>%
9   arrange(desc(punteggio_medio))
10
11 cat("\nTop 10 Generi per Punteggio Medio Rotten Tomatoes:\n")
12 print(head(punteggi_per_genere, 10))
```

Listing 3.5: aggregation based on genre

And then plot the result.

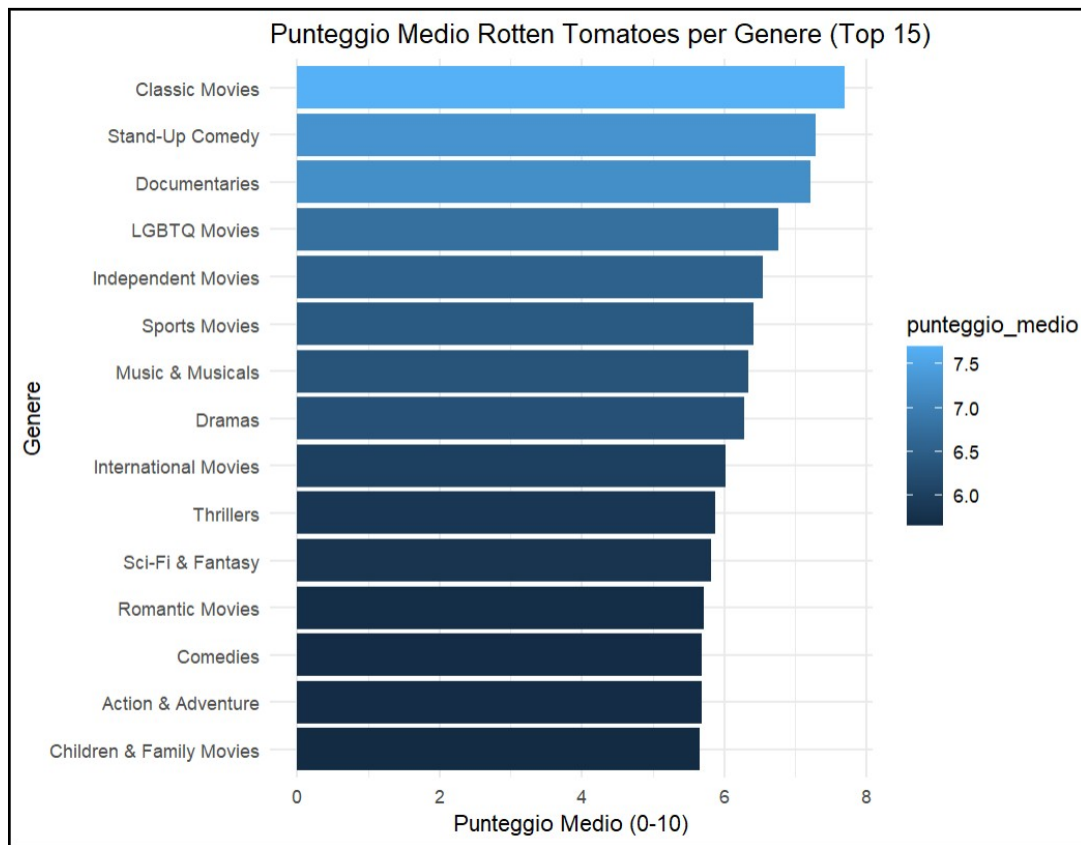


Figure 3.5: Distribution of the mean scores based on genre

3.2.3 Mean scores for release year

A similar work was done in order to find out how the average scores changed in relation to the film's release year:

```

1 punteggi_per_anno <- dataset_finale %>%
2   group_by(anno_rilascio) %>%
3   summarise(
4     punteggi_medio_anno = mean(media_score_rotten, na.rm = TRUE),
5     totale_film = n()
6   ) %>%
7   # Filtra gli anni con meno di 10 film
8   filter(totale_film >= 10)

```

Listing 3.6: aggregation based on release year

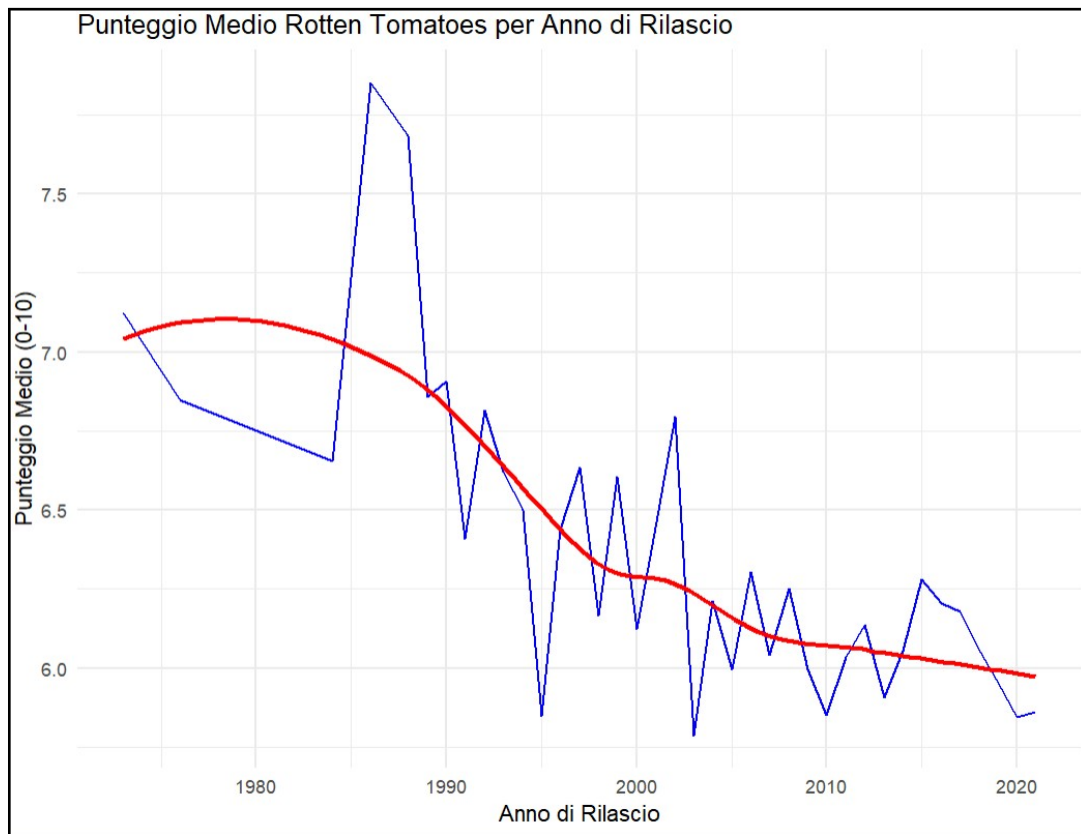


Figure 3.6: Distribution of the mean scores based on release year

3.2.4 Top directors analysis

To find out which directors are the most acclaimed, we had to expand the dataset, as the "director" column often contains multiple entries for a single film:

```
1 dataset_registri_espanso <- dataset_finale %>%
2   # Separa i nomi nella colonna 'director'
3   separate_rows(director, sep = ", ") %>%
4
5   #Rimuovi gli spazi bianchi in eccesso
6   mutate(director = trimws(director)) %>%
7
8   #Rimuovi i record "Unknown"
9   filter(director != "Unknown")
```

Listing 3.7: expansion based on the directors

We went from ~7000 records to ~8000.

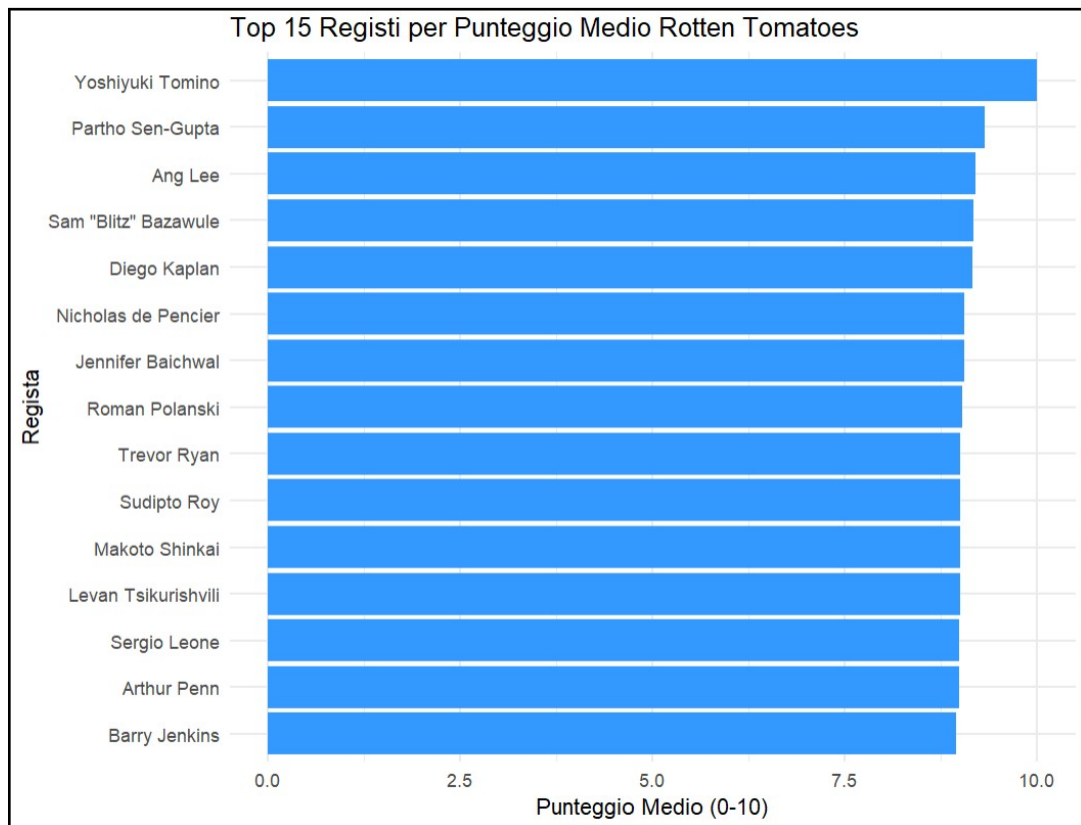


Figure 3.7: Top directors based on average scores

3.2.5 Scores and length analysis

The relationship between film duration and review scores was investigated:

```

1 correlazione_durata_punteggio <- cor(
2   dataset_finale$duration_min,
3   dataset_finale$media_score_rotten,
4   use = "complete.obs" # Ignora le righe con NA
5 )
6
7 cat("Coefficiente di Correlazione (Durata vs. Punteggio):", round(correlazione_
  durata_punteggio, 3), "\n")

```

Listing 3.8: Correlation between scores and length

the correlation coefficient was around 0.03, so that means that there is a weak linear association between the 2 variables. It should be noted that correlation does not imply causation.

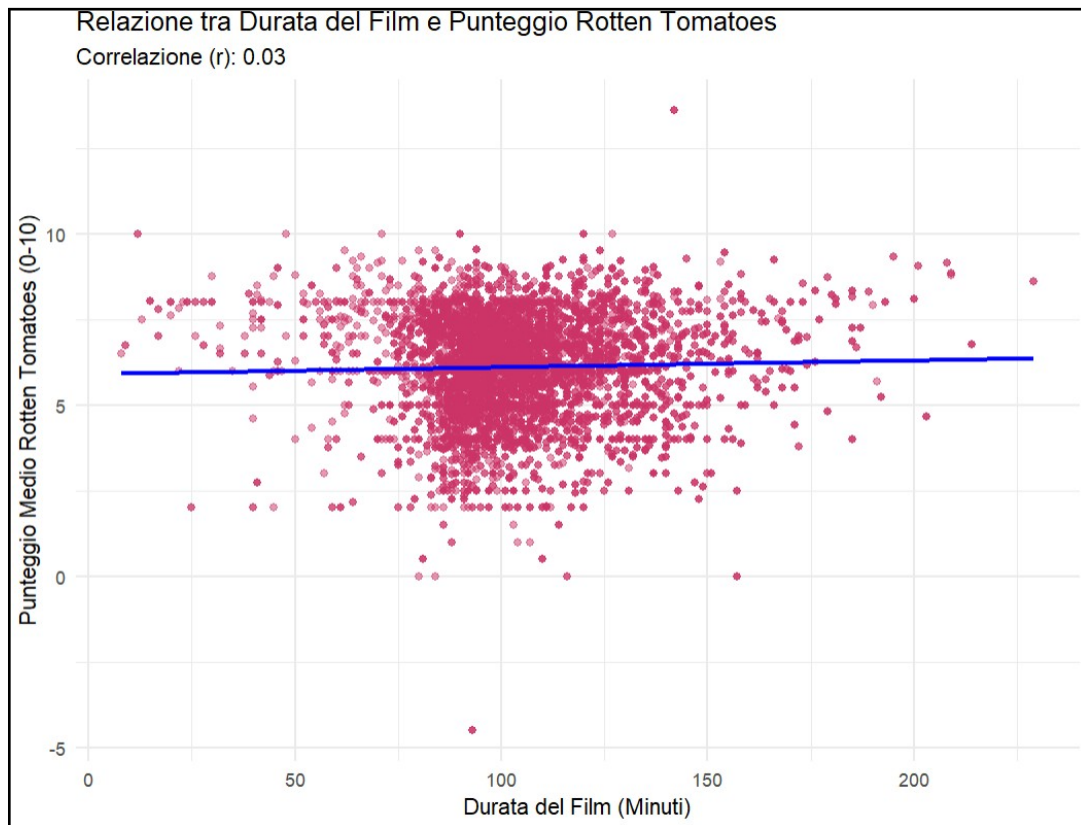


Figure 3.8: Linear correlation between length and scores

3.2.6 Country and scores analysis

The analysis also explored whether specific countries produced films with higher overall ratings. In order to do that, we first need to sum the scores for each country:

```
1 dataset_paese_espanso <- dataset_finale %>%
2   # Separa i nomi nella colonna 'country'
3   separate_rows(country, sep = ", ") %>%
4
5   #Rimuovi gli spazi bianchi in eccesso e i valori 'Unknown'
6   mutate(country = trimws(country)) %>%
7   filter(country != "Unknown")
8
9 cat("Numero di record originali (film unici):", nrow(dataset_finale), "\n")
10 cat("Numero di record espansi per paese:", nrow(dataset_paese_espanso), "\n")
11
12 punteggi_per_paese <- dataset_paese_espanso %>%
13   group_by(country) %>%
14   summarise(
15     punteggio_medio_paese = mean(media_score_rotten, na.rm = TRUE),
16     totale_film = n()
17   ) %>%
18   # Filtra i paesi con poche osservazioni
19   filter(totale_film >= 10) %>%
```

```
20 arrange(desc(punteggio_medio_paese))
```

Listing 3.9: Expansion and grouping based on country

Then we had to use a map and join it with our scores based on country:

```
1 # Scarica i dati del mondo (geometrie)
2 world <- ne_countries(scale = "medium", returnclass = "sf")
3
4 mapping_paesi <- data.frame(
5   tuo_nome = c("United States", "United Kingdom", "South Korea", "China", "
6     India", "Russia", "Taiwan", "Hong Kong", "West Germany", "Czech Republic",
7     "Soviet Union"),
8   nome_mappa = c("United States of America", "United Kingdom", "Republic of
9     Korea", "China", "India", "Russian Federation", "Taiwan", "China", "Germany
10    ", "Czechia", "Russian Federation")
11 )
12 #Applica la mappatura
13 punteggi_mappa <- punteggi_per_paese %>%
14   left_join(mapping_paesi, by = c("country" = "tuo_nome")) %>%
15   mutate(
16     nome_per_join = if_else(is.na(nome_mappa), country, nome_mappa)
17   )
18
19 # Unione dei dati geografici con i punteggi_mappa
20 world_punteggi <- world %>%
21   left_join(
22     punteggi_mappa,
23     by = c
```

Listing 3.10: Join with the map structure

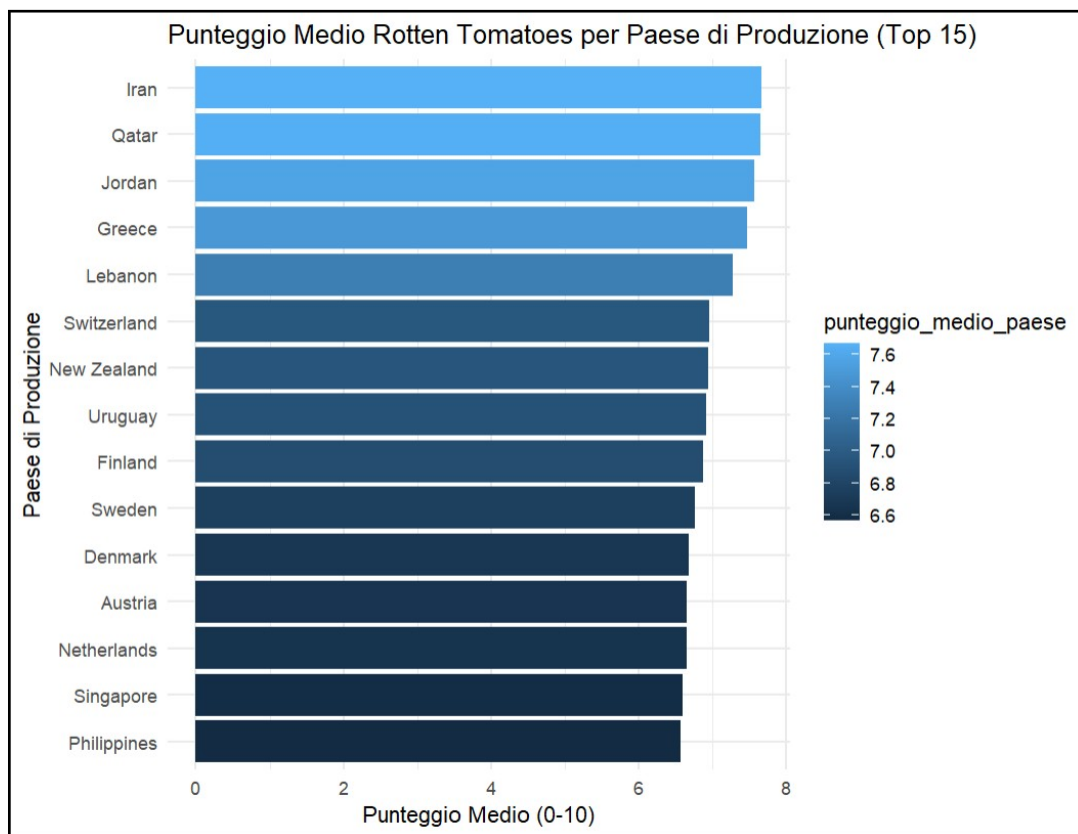


Figure 3.9: top 15 countries with higher scores

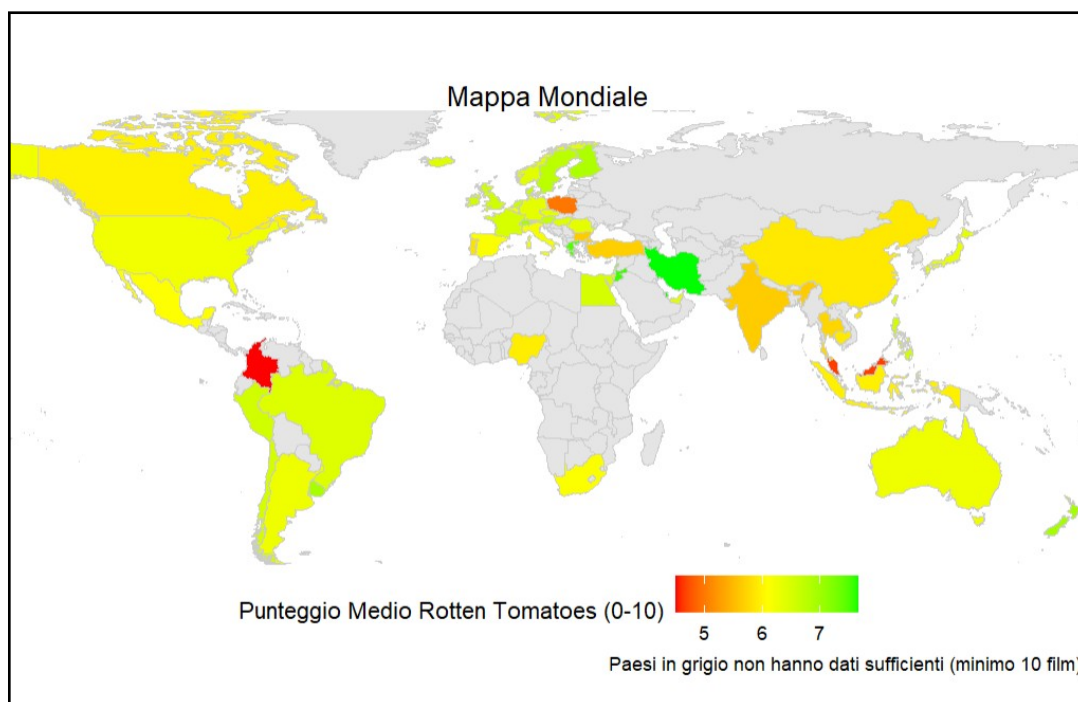


Figure 3.10: Scores based on countries map

3.2.7 Rating and scores correlation

In this section we tried to visualize the correlation between the rating (ex° pg-18) and the mean scores of the reviews:

```
1 punteggi_per_rating_semplificato <- dataset_finale %>%
2   filter(rating_semplificato != "Non Definito") %>%
3   group_by(rating_semplificato) %>%
4   summarise(
5     punteggio_medio_rating = mean(media_score_rotten, na.rm = TRUE),
6     totale_film = n()
7   ) %>%
8   arrange(desc(punteggio_medio_rating))
9
10 cat("\nPunteggio Medio Rotten Tomatoes per Categoria di Rating Semplificato:\n"
11     )
11 print(punteggi_per_rating_semplificato)
```

Listing 3.11: Aggregation based on ratings

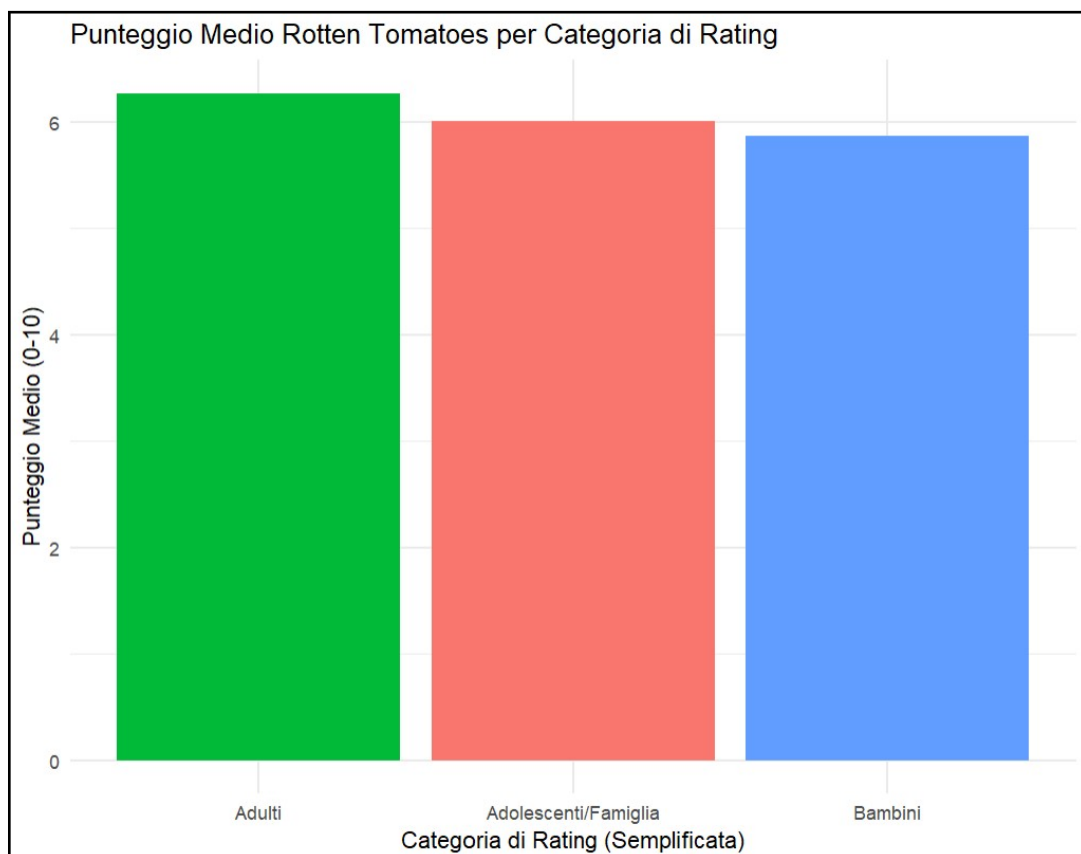


Figure 3.11: Mean scores based on ratings

3.2.8 Actors and scores analysis

We analyzed the correlation between the actors involved in a film and the scores that it received:

```

1 dataset_attori_espanso <- dataset_finale %>%
2   separate_rows(cast, sep = ", ") %>%
3
4   #Rimuovi gli spazi bianchi in eccesso
5   mutate(attore = trimws(cast)) %>%
6
7   filter(attore != "Unknown")
8
9   cat("Numero di record originali (film unici):", nrow(dataset_finale), "\n")
10  cat("Numero di record espansi per attore:", nrow(dataset_attori_espanso), "\n")
11
12 top_attori_puliti <- dataset_attori_espanso %>%
13   group_by(attore) %>%
14   summarise(
15     punteggio_medio = mean(media_score_rotten, na.rm = TRUE),
16     film_contati = n()
17   ) %>%
18   filter(film_contati >= 5) %>%
19   arrange(desc(punteggio_medio))

```

Listing 3.12: Expansion based on the actors involved

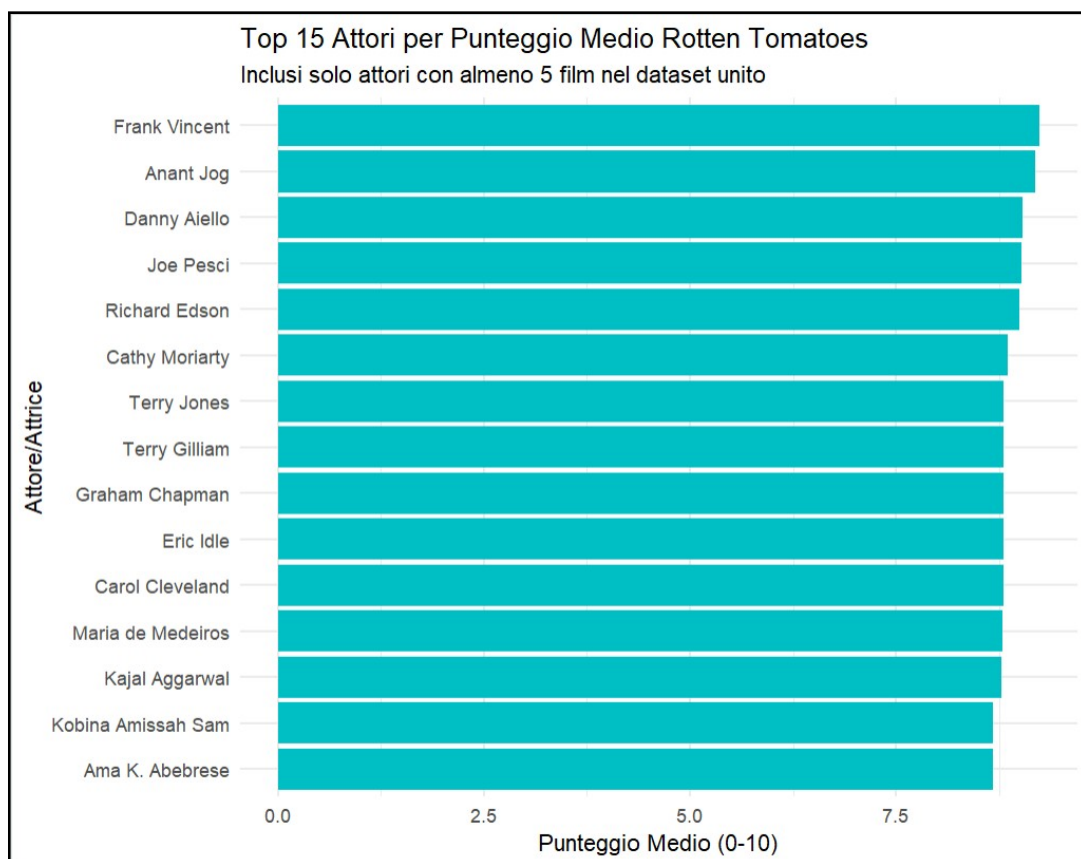


Figure 3.12: top 15 actors with the highest mean scores

Chapter 4

IMDB dataset

4.1 Motivations

After analyzing the correlation between the Netflix and Rotten Tomatoes datasets, we decided to incorporate another dataset to have more data to work with. The IMDB dataset is based mostly on user's reviews, while the Rotten Tomatoes one has professional critics judging the movies. This allows for a comparative study between the similarities and differences that these types of user can have when reviewing a movie.

4.2 Data cleaning

Similarly to what we did before, we are going to transform the dataset to make it easier to work with.

4.2.1 Na count and replacement

To count the number of records with a NA value, we used:

```
1 na_imdb <- imdb %>%
2   summarise(across(everything(), ~ sum(is.na(.)) / n() * 100)) %>%
3   t() %>%
4   as.data.frame() %>%
5   rownames_to_column(var = "Colonna") %>%
6   rename(NA_Percentuale = V1) %>%
7   mutate(NA_Percentuale = round(NA_Percentuale, 2)) %>%
8   arrange(desc(NA_Percentuale))
9
10 print(na_imdb)
```

Listing 4.1: Count of Na values

Notably this dataset doesn't have a single NA value.

4.2.2 Normalization of the scores

Preliminary inspection revealed that ratings in the "score" column were consistently formatted as "x/10"

```
1 # Conversione del rating IMDb a numerico
2 imdb_pulito <- imdb_pulito %>%
3   mutate(
4     rating_solo_numero = stringr::str_remove(rating, "/10"),
5     normalizedScore_imdb = as.numeric(rating_solo_numero)
6   ) %>%
7   drop_na(normalizedScore_imdb)
```

Listing 4.2: normalization of the scores

4.2.3 Normalization of the date

Similarly to what we did before, we also used a unique format for the date:

```
1 #install.packages("lubridate")
2 library(lubridate)
3 if (exists("date", where = .GlobalEnv, inherits = FALSE)) {
4   rm(date)
5 }
6 imdb_pulito <- imdb_pulito %>%
7   mutate(
8     # Il risultato sara automaticamente nel formato YYYY-MM-DD.
9     date_standard = dmy(date)
10  ) %>%
11  select(-date) %>%
12  rename(date = date_standard)
```

Listing 4.3: normalization of the date

4.3 Preparation for the join operation

To prepare the dataset for the join, we first need to normalize the film title (**imdb["movie"]**):

```
1 imdb_join <- imdb_pulito %>%
2   mutate(
3     titolo_normalizzato = tolower(movie),
4
5     titolo_normalizzato = str_replace_all(titolo_normalizzato, "[^a-z0-9]", " "),
6
7     titolo_normalizzato = str_squish(titolo_normalizzato),
8
9     titolo_normalizzato = str_remove_all(titolo_normalizzato, " ")
10  ) %>%
11  filter(titolo_normalizzato != "")
```

Listing 4.4: normalization of the title

Then we aggregated the records based on the title and computed the mean scores:

```
1 imdb_aggregato <- imdb_join %>%
2   group_by(titolo_normalizzato) %>%
3   summarise(
4     media_score_imdb = mean(normalizedScore_imdb, na.rm = TRUE),
5     conteggio_recensioni_imdb = n()
6   ) %>%
7   ungroup()
```

Listing 4.5: Aggregation by title and mean scores

Finally we executed the join operation:

```
1 dataset_unit_3 <- dataset_unit_1 %>%
2   left_join(
3     imdb_aggregato,
4     by = "titolo_normalizzato"
5   )
```

Listing 4.6: Join between IMDB and Netflix/Rotten

4.4 EDA for the IMDB dataset

In this section we explored the properties of the cleaned IMDB dataset.

4.4.1 frequency of the average scores

We plotted the frequencies of the average normalized scores:

```
1 imdb_join %>%
2   ggplot(aes(x = normalizedScore_imdb)) +
3   geom_histogram(binwidth = 0.5, fill = "skyblue", color = "black") +
4   labs(title = "Distribuzione dei Punteggi IMDB", x = "Punteggio Normalizzato",
5         y = "Frequenza")
```

Listing 4.7: frequency based on sscore values

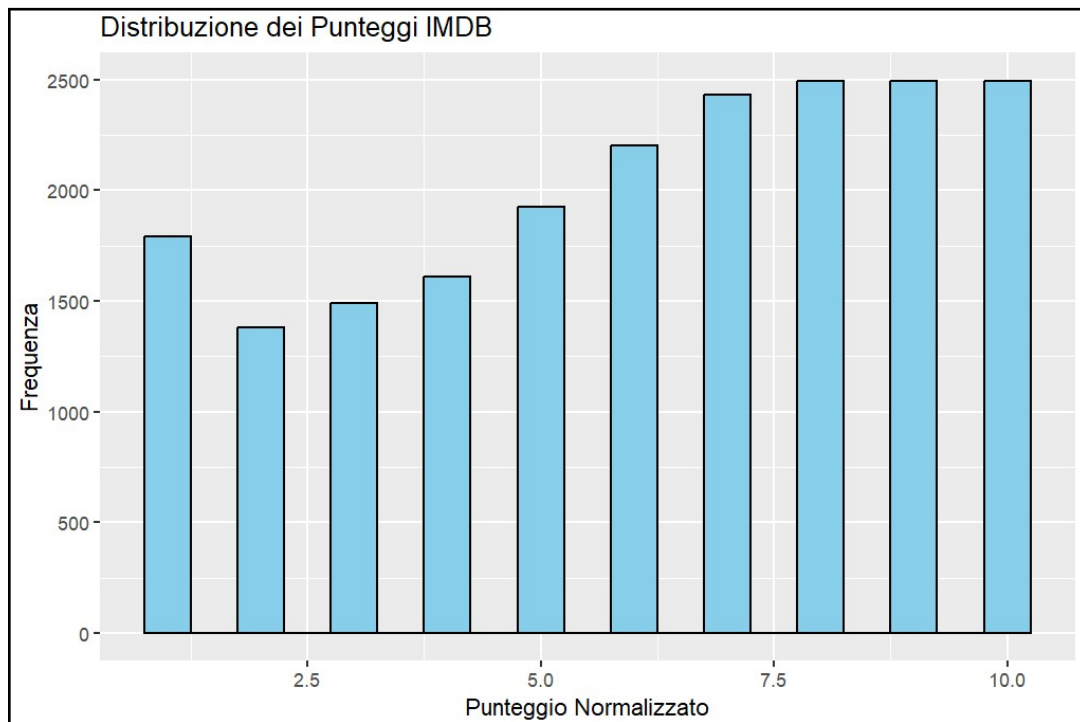


Figure 4.1: Frequency of normalized scores

4.4.2 Temporal distribution of the scores

To investigate potential temporal trends in the ratings, we analyzed the mean scores relative to the review year:

```

1 dati_per_anno <- imdb_join %>%
2   group_by(date) %>%
3   summarise(
4     Media_Rating = mean(normalizedScore_imdb, na.rm = TRUE),
5     Conteggio_Recensioni = n() # Utile per visualizzare l'importanza del punto
6   ) %>%
7   ungroup()

```

Listing 4.8: Aggregation based on year

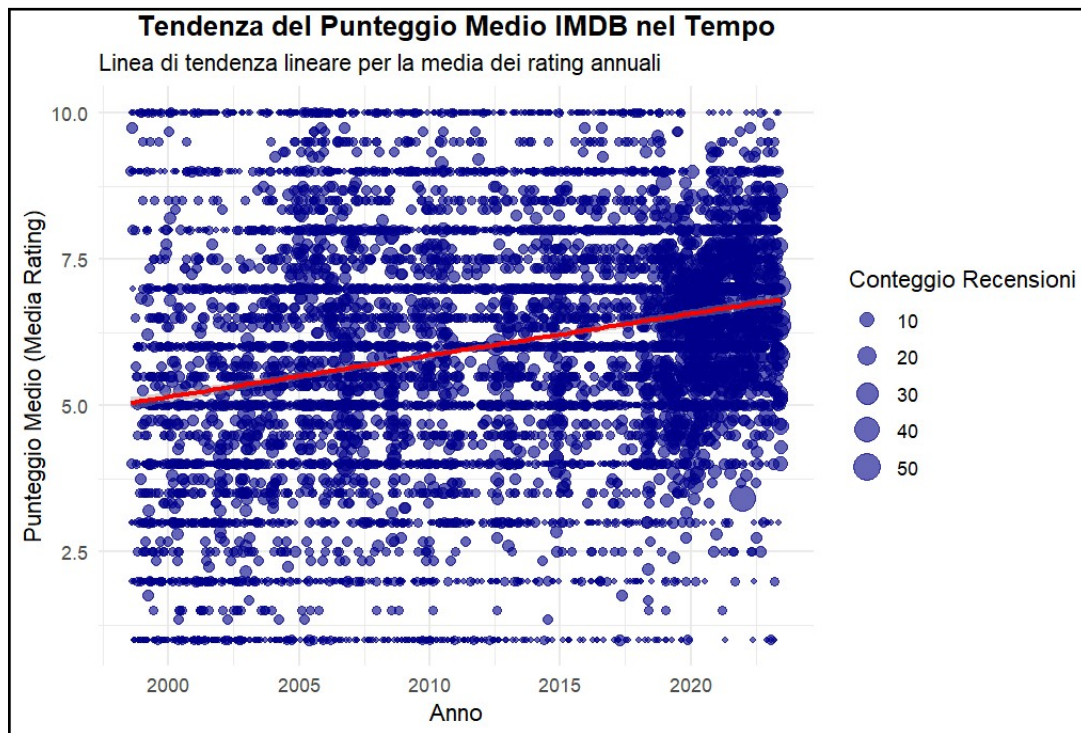


Figure 4.2: Mean scores for each year

Chapter 5

Final dataset analysis

Once the datasets were refined and cleaned, we investigated potential correlations within the data.

This integrated dataset comprises films reviewed by both critics (Rotten Tomatoes) and the general public (IMDb) that are currently available in the Netflix catalogue.

5.1 Na control

Before analyzing the findings, we tried to count how many effective records remained in the join:

```
1 totale_record <- nrow(dataset_finale)
2
3 print(paste("Numero totale di record nel dataset finale:", totale_record))
4
5 colonna_imdb <- "conteggio_recensioni"
6
7 colonna_altri <- "conteggio_recensioni_imdb"
8
9 dataset_unito_3_noNA <- dataset_unito_3 %>%
10   filter(
11     !is.na(!sym(colonna_imdb)) &
12     !is.na(!sym(colonna_altri))
13   )
14
15 conteggio_senza_na <- nrow(dataset_unito_3_noNA)
```

Listing 5.1: Count of effective records

Shockingly, the sample size decreased significantly, from ~7300 records to about 40.

5.2 EDA for the final dataset

5.2.1 Top films with different scores

To identify the films with the most pronounced discrepancies between the two evaluation platforms:

```
1 top_10_popolari_clean <- dataset_unit_3_noNA %>%
2   group_by(titolo_normalizzato, titolo_film) %>%
3   summarise(
4     voto_rotten = mean(media_score_rotten, na.rm = TRUE),
5     voto_imdb_100 = mean(media_score_imdb, na.rm = TRUE),
6     recensioni_RT = mean(conteggio_recensioni, na.rm = TRUE),
7     recensioni_IMDb = mean(conteggio_recensioni_imdb, na.rm = TRUE),
8     .groups = "drop"
9   ) %>%
10  mutate(
11    totale_popolarita = recensioni_RT + recensioni_IMDb,
12    differenza = voto_rotten - voto_imdb_100
13  ) %>%
14  arrange(desc(totale_popolarita)) %>%
15  head(10)
```

Listing 5.2: Top films with biggest score difference

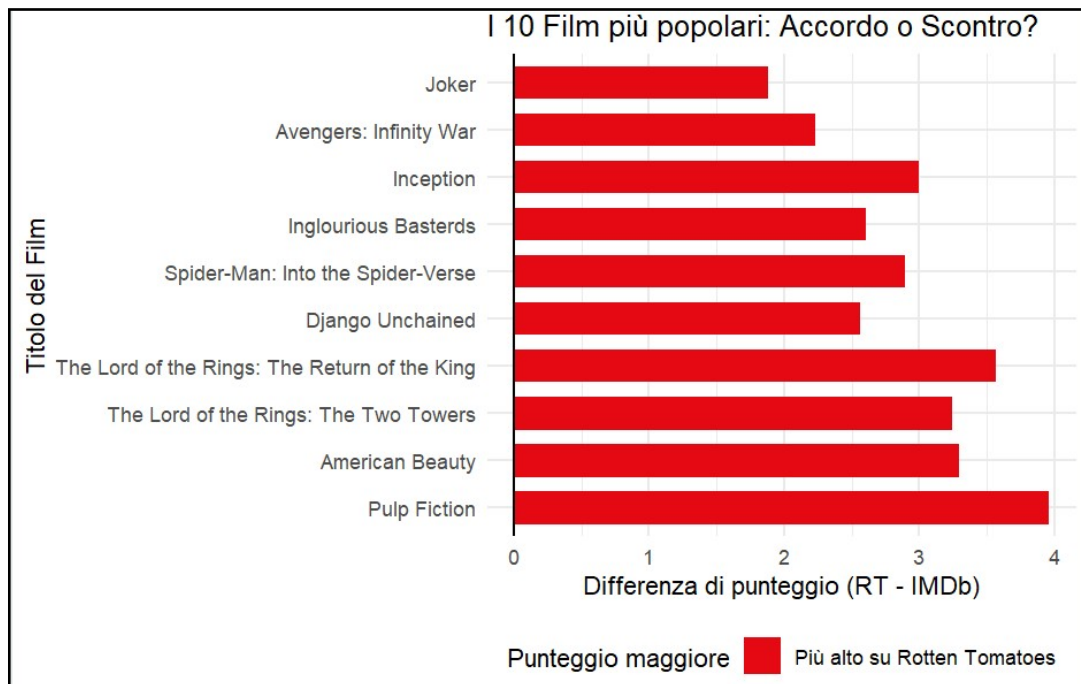


Figure 5.1: Top films with different scores

5.2.2 Top genres with different scores

Similarly, an analysis was conducted to identify genres exhibiting the largest disparity in ratings:

```

1 analisi_generi <- dataset_unit_3_noNA %>%
2   mutate(diff_singola = media_score_rotten - media_score_imdb) %>%
3   group_by(listed_in) %>%
4   summarise(
5     differenza_media = mean(diff_singola, na.rm = TRUE),
6     numero_film = n(),
7     .groups = "drop"
8   ) %>%
9   filter(numero_film >= 3)

```

Listing 5.3: Top genres with biggest score difference

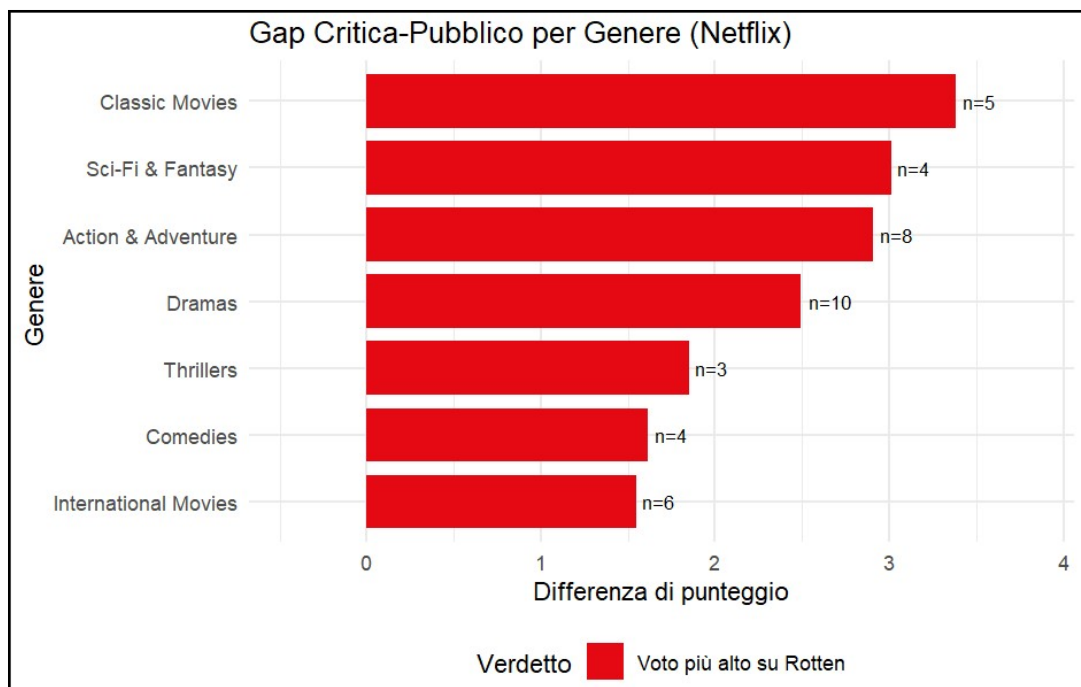


Figure 5.2: Top genres with different scores

5.2.3 Genres and origin country correlation

We tried to find if there were any genres produced by a specific country that had a reasonable amount of good reviews, in order to do that we first need to expand the records on the **["country"]** and **["listed_in"]** columns:

```

1 nicchie_df <- dataset_unit_3_noNA %>%
2   # Separa i Paesi
3   separate_rows(country, sep = ", ") %>%
4   mutate(country = trimws(country)) %>%
5   # Separa i Generi
6   separate_rows(listed_in, sep = ", ") %>%
7   mutate(listed_in = trimws(listed_in)) %>%
8   # Filtri di pulizia base
9   filter(country != "Unknown", country != "", listed_in != "")

```

Listing 5.4: Expansion of country and listed in

Then we had to compute the scores for each combination:

```

1 nicchie_stats <- nicchie_df %>%
2   group_by(country, listed_in) %>%
3   summarise(
4     media_rotten = mean(media_score_rotten, na.rm = TRUE),
5     media_imdb = mean(media_score_imdb, na.rm = TRUE),
6     numero_film = n(),
7     .groups = "drop"
8   ) %>%
9   mutate(score_combinato = (media_rotten + media_imdb) / 2) %>%
10  filter(numero_film >= 1) %>%
11  arrange(desc(score_combinato))

```

Listing 5.5: heatmap computation

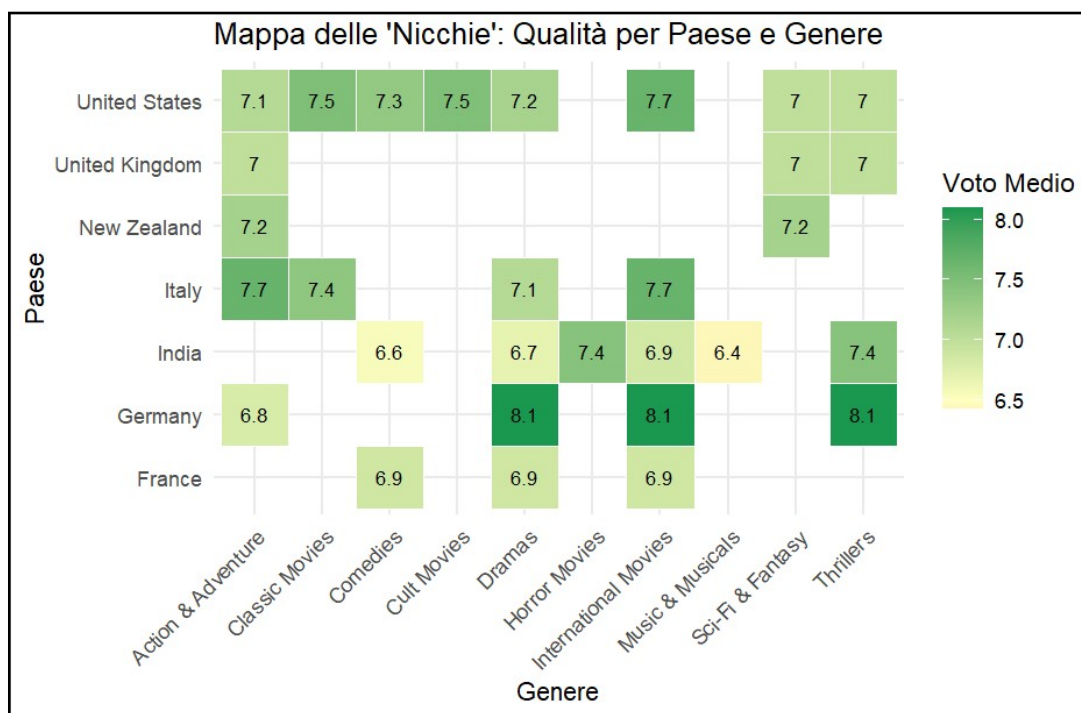


Figure 5.3: Heatmap for country and genres

5.2.4 Length sweet spot difference

We computed the mean ratings based on the length of the movies between the 2 platforms:

```

1 dati_confronto <- dataset_unit_3_noNA %>%
2   select(titolo_normalizzato, duration_min, media_score_rotten, media_score_
3     imdb) %>%
4   distinct() %>%
5   filter(duration_min >= 60 & duration_min <= 210)
6 #Trasformazione in formato 'Long' (Lungo)
7

```

```

8 dati_confronto_long <- dati_confronto %>%
9   pivot_longer(
10     cols = c(media_score_rotten, media_score_imdb),
11     names_to = "Piattaforma",
12     values_to = "Punteggio"
13   ) %>%
14   mutate(Piattaforma = case_when(
15     Piattaforma == "media_score_rotten" ~ "Rotten Tomatoes",
16     Piattaforma == "media_score_imdb" ~ "IMDb"
17   ))

```

Listing 5.6: Ratings based on length computation

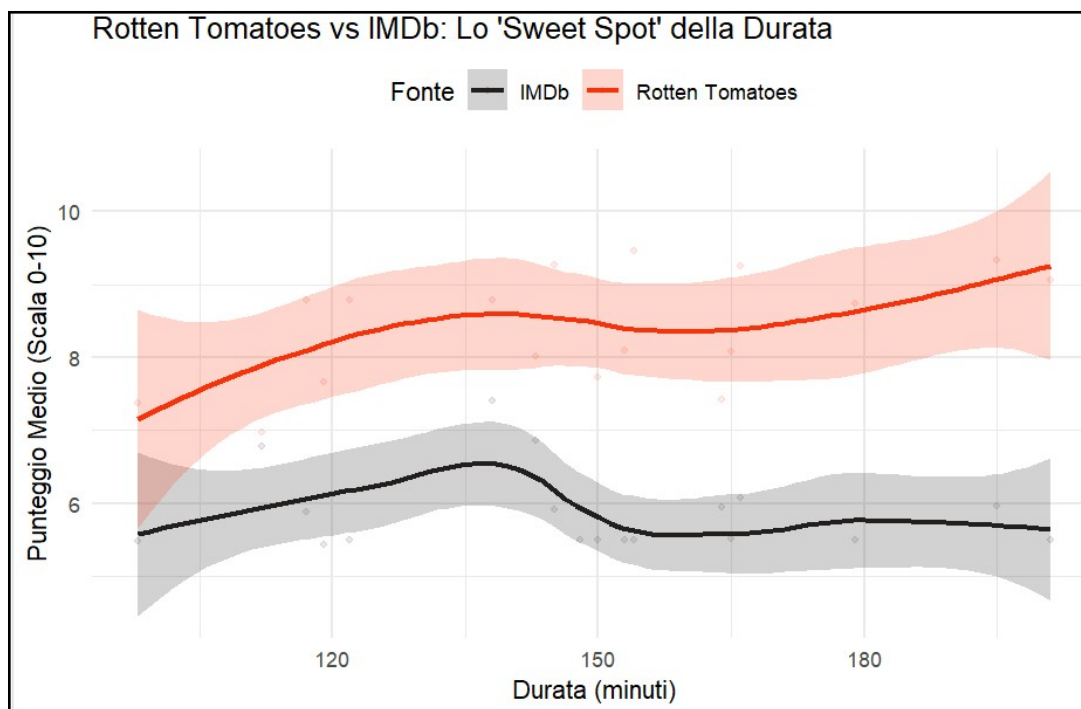


Figure 5.4: Comparison of the "sweet spot" duration between platforms

Chapter 6

Conclusion

The present study conducted a comprehensive cross-platform analysis by integrating datasets from Netflix, Rotten Tomatoes, and IMDb to investigate the correlation between content availability and audience/critic reception. By leveraging RStudio for data cleaning, merging, and statistical modeling, several key insights have emerged regarding the digital cinematic landscape.

6.1 Summary of findings

Among all of the correlations we found, the key findings include:

1. **Higher ratings:** the critics tend to assign higher mean ratings on most of the films, than the general users.
2. **Genre trends:** there is a significant difference between the mean ratings for classic movies, Sci-fi & fantasy, action & adventure genres between the platforms.
3. **Niche production:** the highest-rated combinations of origin country and genre were identified as German dramas and international films.
4. **Sweet spot duration:** there is a well defined variance between the ratings based on the film's duration between the two platforms.

6.2 Limitations of the study

Despite the insights gained from this multi-platform analysis, several limitations must be acknowledged:

- **Data Attrition and Sample Size:** The most significant constraint was the substantial reduction in the number of records following the merging process. The low intersection rate between the Netflix catalog and the ratings from Rotten Tomatoes and IMDb resulted in a diminished final

dataset.

This attrition may limit the statistical power of the correlations and the generalizability of the findings to the entire Netflix library.

- **Metadata Inconsistency:** The lack of a standardized unique identifier (such as an ISAN or IMDb ID) across all three datasets required merging based on movie titles.
Discrepancies in naming conventions, punctuation, and release dates between platforms likely led to a modest match rate, potentially excluding relevant titles that were formatted differently in separate databases.
- **Categorical Aggregation:** To facilitate a coherent analysis, the `listed_in` column underwent a process of consolidation. While this allowed for a broader comparison of genres, such aggregation may overlook the nuances of sub-genres and niche categories that are specific to the Netflix internal tagging system.
- **Selection Bias:** The study is inherently biased toward high-profile titles that possess sufficient visibility to be indexed and reviewed across multiple platforms. Consequently, independent or international films with lower marketing reach may be underrepresented in the final merged output.